

Community Security Alert System (CSAS): Systems Design — Workshop No. 2

Felipe Jose Garzon Herrera, 20251020132, Juan Esteban Quintero Gordillo, 20251020137,
Henry Samuel Garrido Medina, 20251020125, Gabriel Mateo Cusba Marin, 20251020128

*Systems Analysis & Design, Semester 2026-I
Eng. Carlos Andrés Sierra, M.Sc.*

Computer Engineering Program, Universidad Distrital Francisco José de Caldas, Bogotá D.C., Colombia

Abstract—This document presents the systems design specification for the Community Security Alert System (CSAS), a crowd-sourced mobile and web platform engineered to improve safety at Universidad Distrital Francisco José de Caldas. The design builds directly on the empirical findings of Workshop No. 1, which identified three critical gaps in the current campus security posture: a persistent under-reporting culture, a spatial and temporal concentration of incidents in specific zones and hours, and the absence of a centralized real-time reporting tool. The proposed architecture adopts a microservices-based, event-driven approach organized across five layers. Key design decisions include a hybrid AI-and-human verification pipeline, geofenced alert dispatch with zone-weighted and time-weighted priority logic, and a community engagement module targeting the adoption gap. By integrating a mobile-first reporting interface with automated verification and a multi-channel notification engine, the system is designed to reduce average emergency response times from 11.4 minutes to under 5 minutes.

Index Terms—campus security, crowd-sourced incident reporting, microservices architecture, real-time notifications, geofenced alerts, verification pipeline, event-driven design.

1. INTRODUCTION

The CSAS project addresses a systemic failure in traditional security reporting at Universidad Distrital Francisco José de Caldas. Workshop No. 1 established, through six independent data collection methods, that the current campus security posture suffers from three compounding problems. First, most incidents go unreported through official channels because the friction of current reporting tools is too high. Second, incidents are spatially and temporally concentrated: all observed suspicious events occurred during evening or night hours, and the parking lot was consistently identified as the highest-risk zone. Third, there is no centralized, real-time incident management tool; security staff rely on phone calls and walk-up reports, which introduces communication delays averaging 11.4 minutes per incident.

The goal of this design phase is to translate those analytical findings into a coherent technical architecture that directly addresses each identified gap. Table 1 summarizes the operational difference between the current state and the design targets established for this workshop.

Table 1. Operational Gap Analysis: Current State vs. Design Targets

Dimension	Baseline	Target	Gap
Response time	11.4 min	<5 min	−6.4 min
Reporting rate	41%	>85%	+44 pp
User adoption	~12%	>60%	+48 pp
Delivery success	~87%	>99.9%	+12.9 pp

The remainder of this document is organized as follows. Section II defines functional and non-functional requirements. Section III describes the system architecture. Section IV presents the implementation strategy. Section V addresses

risk management. Section VI defines performance metrics and testing. Section VII discusses ethical considerations. Section VIII concludes.

2. REQUIREMENTS ANALYSIS

A. Functional Requirements

The following requirements are derived directly from Workshop No. 1. Each is traceable to at least one empirical finding.

- **FR-01 (Incident Reporting):** Users shall report incidents by specifying type, GPS location, and description. Addresses the under-reporting barrier.
- **FR-02 (Geofenced Notifications):** Real-time alerts shall be broadcast to users within a 500 m radius of a verified incident. Responds to the spatial concentration of incidents in high-risk zones.
- **FR-03 (AI Verification):** An AI plausibility scorer shall perform initial metadata validation and noise filtering. Mitigates data fragmentation in current manual records.
- **FR-04 (Administrative Dashboard):** Security personnel shall have a web interface to review, escalate, or resolve active alerts. Replaces fragmented phone and walk-up reporting.
- **FR-05 (Crowd-Sourced Validation):** Users shall confirm or flag reports submitted by others. Leverages the high willingness-to-participate rate observed in the student survey.
- **FR-06 (Automatic Geolocation):** The mobile client shall automatically capture GPS coordinates for every submitted report. Based on spatial precision needs identified in direct observation sessions.
- **FR-07 (Centralized Database):** A persistent incident log shall be maintained for historical pattern analysis. Addresses the current state in which incidents go unrecorded.
- **FR-08 (Emergency SOS):** A one-touch emergency button

shall trigger immediate security dispatch. Targets a reduction in response time of more than half the current baseline.

- **FR-09 (Analytics Engine):** The system shall generate periodic security heatmaps for university management. Provides data-driven oversight currently absent.
- **FR-10 (Media Attachments):** Users shall attach image or video evidence to reports to improve verification quality.

B. Non-Functional Requirements

- **NFR-01 (Latency):** End-to-end latency from report submission to alert broadcast must be under 30 s for 95% of incidents.
- **NFR-02 (Reliability):** The notification engine shall maintain a 99.9% delivery success rate across all channels.
- **NFR-03 (Availability):** The system must maintain 99.9% uptime, with guaranteed operation during high-risk hours (18:00–22:00).
- **NFR-04 (Privacy/Security):** All data must be encrypted via TLS 1.3 and comply with Ley 1581 de 2012.
- **NFR-05 (Scalability):** The API gateway must handle a 300% surge in concurrent requests without performance loss.
- **NFR-06 (Usability):** The reporting workflow must require no more than three interactions to submit an alert.
- **NFR-07 (Maintainability):** Any microservice shall be updatable or replaceable independently.
- **NFR-08 (Data Integrity):** No alert shall broadcast as verified without at least two community confirmations or one administrative approval.

3. SYSTEM ARCHITECTURE

A. Architectural Style

The CSAS adopts a microservices-based architecture in which the system is decomposed into independent components that communicate through well-defined interfaces. This choice is driven by three characteristics of the problem: the verification, notification, and analytics subsystems have independent load profiles and scaling needs; alert dispatch must be decoupled from incident submission to prevent cascading delays during high-volume periods; and the system must integrate with external actors through stable, versioned interfaces.

B. System Layers

The architecture is organized across five layers:

- **Presentation:** Mobile application and web portal for incident submission and alert reception.
- **Application:** API gateway handling authentication, rate limiting, and microservice routing.
- **Service:** Five domain microservices (Section III-C).
- **Data:** Persistent storage for incidents, user profiles, media, audit logs, and analytics, backed by a Redis cache for low-latency reads.
- **Integration:** Interfaces to SIURE UD, local authorities, and the SMS gateway.

C. Microservices Design

Table 2 describes the five core microservices. Each publishes and consumes domain events over the message broker (RabbitMQ), ensuring loose coupling across the service layer.

Table 2. Core Microservices and Responsibilities

Service	Responsibility
Incident	Receives, validates, and persists reports. Enriches each event with geospatial metadata and publishes <i>IncidentCreated</i> .
Verification	Orchestrates the verification workflow using an adaptive threshold algorithm and AI plausibility scorer. Supports multi-user confirmation, administrative override, and priority bypass for high-severity types. Publishes <i>IncidentVerified</i> .
Dispatcher	Computes the notification target set using zone-weighted and time-weighted priority logic. Distributes alerts via push and SMS fallback.
User	Manages registration, roles, and notification preferences. Implements alert fatigue mitigation by dampening non-critical alerts for high-dismissal users.
Analytics	Consumes domain events to populate the data warehouse. Produces security dashboards and periodic heatmap reports (FR-09).

D. External Integrations

Table 3. External Integrations

System	Type	Description
SIURE UD	REST API	Bidirectional bridge: CSAS forwards verified incidents; SIURE UD provides historical baseline. Complements the existing platform without replacing it.
Authorities	REST API	Secure endpoint activated for high-severity incidents requiring external intervention. Modeled after the Policía Nacional / Vigila App integration.
SMS Gateway	Ext. service	Fallback channel for users without push capability (NFR-02). Auto-activates after a 10-second push delivery timeout.

Figs. 1 and 2 provide visual representations of the architecture and main process flow described in Sections III-B through III-D.

4. TECHNICAL IMPLEMENTATION STRATEGY

A. Technology Stack

Table 4 presents the recommended technology stack. Each choice is justified with reference to the requirements defined in Section II.

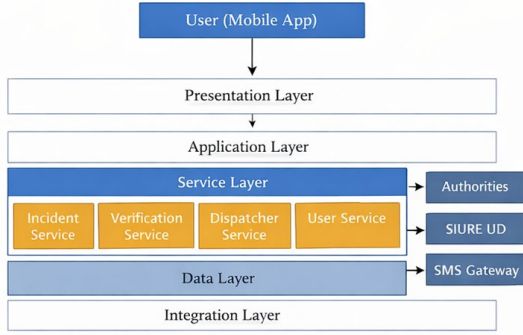


Figure 1. CSAS System Architecture Diagram. The REST API gateway acts as the central connectivity node between the presentation, service, and data layers.

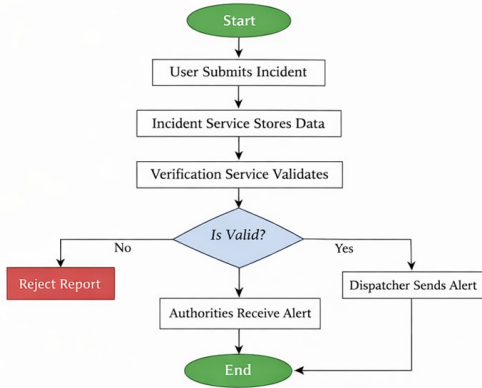


Figure 2. Main Process Flow Diagram. Information flows from user submission through the verification pipeline to alert dispatch and database logging.

B. Implementation Phases

Phase 1 — Planning and Architecture (Month 1). Definition of the final architecture, validation of all requirements, and development environment setup. *Success criterion:* all FR and NFR are traceable to Workshop No. 1 findings. *Deliverable:* architecture diagrams and requirements documentation.

Phase 2 — Development (Month 2). Implementation of frontend, backend microservices, database schema, and messaging infrastructure. *Success criterion:* end-to-end prototype in which a user can submit a report (FR-01) and receive a geofenced alert (FR-02). *Deliverable:* functional prototype.

Phase 3 — Testing and Validation (Month 3). Execution of the full testing strategy (Section VI-B). *Success criterion:* system meets NFR-01 (<30 s) and NFR-02 (99.9%) under simulated load. *Deliverable:* test reports.

Phase 4 — Deployment and Monitoring (Month 4). Cloud deployment, external integrations, and production monitoring activation. *Success criterion:* 99.9% uptime under real-world conditions. *Deliverable:* deployed system with live dashboard.

Table 4. Recommended Technology Stack

Component	Technology	Justification
Mobile app	React Native	Single codebase for Android/iOS; supports ≤ 3 -interaction reporting (NFR-06).
API gateway	Node.js / Nginx	High I/O throughput for concurrent submissions; handles routing and rate limiting.
Backend services	Node.js	Asynchronous, event-driven I/O supports NFR-01 (<30 s latency).
Verification Engine	Python / FastAPI	Python's ML ecosystem simplifies integration of the AI plausibility scorer.
Message broker	RabbitMQ	Priority queue support; decouples ingestion from processing to mitigate volume spikes.
Database	PostgreSQL + PostGIS	Native geospatial queries for zone-weighted routing and heatmaps (FR-06, FR-09).
Push notifications	Firebase FCM	Cross-platform delivery; supports 99.9% delivery target (NFR-02).
SMS fallback	Twilio API	Guaranteed delivery for users without smartphone access; supports equity of access.
Infrastructure	AWS / Azure	Container orchestration via Kubernetes enables auto-scaling (NFR-03, NFR-05).

5. RISK AND COMPLEXITY MANAGEMENT

A. Sensitivity Parameter Mitigation

Workshop No. 1 identified five operational parameters with the strongest influence on CSAS behavior. Left unaddressed, each represents a distinct failure path. Table 5 maps each parameter to its design mechanism.

B. Feedback Loops and Emergent Behavior

Reinforcing loop. The system exhibits a positive reinforcing dynamic: useful alerts increase user vigilance, which increases reporting, which generates more alerts, progressively improving system effectiveness as adoption grows [3]. The community engagement module (Phase 3) targets the critical adoption threshold of approximately 20–25% of the campus population, the point at which report density sustains the verification mechanism without heavy reliance on security staff.

Balancing loop. A counterbalancing dynamic operates in the opposite direction: excessive alerts cause users to ignore them (alert fatigue), reducing system effectiveness over time [3]. The alert fatigue mitigation module in the User Service damp-

Table 5. Sensitivity Parameter Risk and Design Mitigation

Parameter	Risk	Design Mechanism
Report volume spike	Verification engine saturates; alert latency exceeds targets.	RabbitMQ decouples report ingestion from verification processing. Auto-scaling policies on the Verification Engine pods. Priority queue lanes for high-severity events that bypass the standard queue.
Low user participation	Real incidents go undetected; system loses credibility.	Low-friction reporting interface (FR-01, ≤ 3 steps). Anonymous reporting option. Community engagement module (Phase 3). Integration with SIURE UD as proof of institutional endorsement.
Night-time concentration	Higher alert volume during off-hours; fewer moderators available.	Time-weighted priority logic elevates night-hour alert priority. On-call escalation path notifies the on-duty security team when no active moderators are available.
High-risk zone clustering	Alert storms in parking lot and dormitory zones saturate users.	Zone-based rate limiting on alert broadcast. Spatial clustering algorithm aggregates co-located simultaneous reports into a single composite alert to prevent redundancy.
Adverse weather conditions	Campus movement decreases; visibility drops; incident risk rises.	Weather API integration feeds real-time conditions into the risk scoring model. Adverse weather triggers automatic elevation of monitoring sensitivity in high-risk zones.

ens non-critical notifications for users with high dismissal rates. Simultaneously, the Verification Engine’s quality filters prevent low-confidence reports from reaching the dispatcher. Calibrating the right confirmation threshold — fast enough for genuine emergencies, strict enough to suppress noise — is the most critical design tension in the system.

Two non-linear effects are also worth noting: the relationship between adoption and report volume is non-linear (a small user increase can produce a disproportionate volume surge), and alert activity is not uniformly distributed across campus zones or time of day. For this reason, the dispatcher applies zone-weighted and time-weighted priority logic rather than treating all reports equally [2].

C. Failure Mode Analysis

Table 6 maps each major architectural dependency to a failure mode, its impact, and the corresponding mitigation.

6. PERFORMANCE AND QUALITY ASSURANCE

A. Key Performance Indicators

Table 7 defines the measurable targets against which the implemented system will be evaluated. Each KPI is aligned with a specific requirement from Section II.

B. Testing Strategy

Unit testing. Validates individual components in each microservice. Each service shall maintain at least 80% code coverage.

Integration testing. Verifies the full event chain: Incident Service → Verification Engine → Alert Dispatcher. Tests run against a staging environment with seeded campus zone data.

Load testing. Evaluates system behavior under a 300% traffic surge (NFR-05). Target: zero timeout errors and latency below 30 s at the 95th percentile under full load.

User acceptance testing (UAT). A pilot group of students and security staff validates usability and correctness under realistic conditions. Success criterion: at least 85% of participants complete a report submission in under two minutes.

Security testing. Verifies compliance with NFR-04 and Ley 1581 de 2012: data encryption (TLS 1.3), secure authentication, and access control mechanisms.

7. ETHICAL AND SAFETY CONSIDERATIONS

A. Privacy and Data Protection

The CSAS collects three categories of personal data: registration information (name, institutional email, and campus role), real-time location data generated during incident submission, and incident media including photographs and video. Each category carries distinct privacy obligations under Colombian law.

All data handling must comply with Ley 1581 de 2012, Colombia’s general personal data protection regime [6]. Four obligations are directly binding on the CSAS. First, users must provide explicit, informed consent at registration, specific to the purpose of campus safety. Second, collected data may only be processed for declared safety purposes and cannot be transferred to third parties or used for advertising. Third, every user retains the right to access, correct, and request deletion of their data, and the system must provide a mechanism for exercising these rights. Fourth, incident records should be subject to a defined retention policy — a maximum of twelve months is recommended — after which records are deleted or irreversibly anonymized.

For sensitive incident types such as harassment or threats, the system provides an anonymous reporting option. The reporter’s identity is encrypted at rest and accessible only to system administrators under a documented, access-controlled protocol; every access is recorded in the immutable audit log. A formal data processing agreement with the university’s legal

Table 6. Failure Mode Analysis and Mitigation Strategies

Failure Mode	Impact	Mitigation
API Gateway outage	All user submissions fail; system is unreachable.	Multi-region deployment with health checks and automatic failover. Mobile app queues reports locally and retries on reconnection.
Verification Engine overload	Alert latency exceeds NFR-01.	Auto-scaling on Verification Engine pods. High-severity incidents bypass AI scoring via a priority dispatch path.
Push notification failure	Users do not receive critical alerts.	SMS fallback via Twilio activates after a 10-second delivery timeout. Delivery receipts confirm end-to-end success.
Database unavailability	Risk of incident data loss.	PostgreSQL primary-replica replication. Automated daily backups. Write-ahead logging ensures no committed data is lost.
AI scorer false positive	Legitimate incidents blocked before dispatch.	Human moderator review queue for all low-confidence scores. Configurable override timer (default: 90 s) escalates to human review.
Mobile connectivity dead zone	Submissions fail in low-coverage campus areas.	Offline queuing in the mobile app. SMS submission as an alternative channel. A connectivity audit is recommended prior to Phase 1 infrastructure planning.

Table 7. Key Performance Indicators

KPI	Target	Method	Freq.
Alert latency (end-to-end)	<30 s (NFR-01)	System logs	Daily
Delivery success rate	>99.9% (NFR-02)	Notif. logs	Daily
System availability	99.9% (NFR-03)	Monitoring	Monthly
Incident reporting rate	>85% logged	DB records	Weekly
User adoption rate	>60% of students	App analytics	Monthly
Verification latency	<60 s	Event logs	Daily
Alert accuracy	≥2 confirm. or 1 admin	Validation	Daily
Scalability under surge	300% surge (NFR-05)	Load tests	Monthly

counsel is a prerequisite to production deployment [6].

B. Equity and Accessibility

A mobile-first design must not create an equity gap for campus members who lack a smartphone or reliable mobile data. The multi-channel notification architecture (NFR-02) includes an SMS gateway as both a delivery and submission fallback, ensuring that all users can send and receive security information regardless of device access. The web portal provides an additional access path for WiFi-connected users. All user-facing components use Spanish as the primary language, and the onboarding flow must be completable in under two minutes (NFR-06). The system should not presuppose advanced digital literacy.

C. Prevention of Misuse

Three misuse vectors are explicitly addressed. Intentional false reports are mitigated by the Verification Engine and human moderation layer (Section III-C). Harassment through the platform is deterred by the immutable audit log, which records all moderation decisions and provides accountability for every consequential action. The anonymous reporting option includes rate limiting to prevent a single user from submitting an excessive number of anonymous reports within a configurable time window. These mechanisms ensure that the CSAS cannot be used to harm the community it is designed to protect.

8. CONCLUSION AND NEXT STEPS

A. Design Summary

This document translates the empirical findings of Workshop No. 1 into a complete architectural blueprint for the CSAS. Each of the three core problems identified through six independent data collection methods has a direct and traceable design response: the reporting culture gap is addressed by the low-friction mobile interface and community engagement module; the non-uniform incident distribution is addressed by the adaptive zone-weighted and time-weighted dispatch logic; and the operational performance gaps are formalized as KPI targets in Section VI.

The design is guided by five systems engineering principles: *modularity* (independent microservices), *scalability* (per-component auto-scaling), *maintainability* (containerized CI/CD deployment), *reliability* (redundancy and offline queuing), and *privacy* (compliance-by-design with Ley 1581 de 2012). Every major design decision is traceable to a specific finding from Workshop No. 1.

B. Open Assumptions Requiring Validation

Willingness to report. The design assumes that users will submit reports if friction is sufficiently reduced. Work-

shop No. 1 survey data provides directional support (all incident-experienced participants would use the app), but this has not been confirmed under real operating conditions. It will be tested in the Phase 3 UAT.

Verification latency. The design assumes that the verification mechanism can operate within a 45-second latency budget. This depends on implementation-specific choices not yet finalized and will be validated through Phase 4 load testing.

Mobile connectivity coverage. The system assumes reliable connectivity across all campus zones. The observation phase of Workshop No. 1 did not assess signal coverage systematically. A campus-wide connectivity audit is recommended before Phase 1 infrastructure planning.

C. Recommendations for Workshop No. 3

Prioritize Phase 1. Deploying the minimum viable reporting loop will generate real operational data to validate the sensitivity parameters identified in Workshop No. 1.

Conduct a connectivity audit. Assess WiFi and mobile signal coverage across the four zones observed during Workshop No. 1 before finalizing the notification delivery architecture.

Expand the student survey. A minimum sample of 200 respondents (stratified by faculty and time of day) is needed to validate the 35% incident prevalence estimate from the 20-student Workshop No. 1 survey.

Establish a formal agreement with SIURE UD. A data-sharing agreement is a prerequisite for the Section III-D integration; it should be initiated early, as institutional approval typically requires significant lead time.

Engage university legal counsel. A formal data processing agreement under Ley 1581 de 2012 and official institutional endorsement are both prerequisites for reaching the 60% community adoption target and must be pursued in parallel with Phase 1 development.

Consistent with the principle that design is iterative, this document represents a design baseline, not a final specification. All decisions are subject to revision as implementation proceeds and real data becomes available.

8. REFERENCES

- [1] C. A. Sierra, *TeamWork as Computer Engineers — CS Collaboration Guidelines*. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.
- [2] P. Checkland, *Systems Thinking, Systems Practice*. New York: John Wiley & Sons, 1981.
- [3] D. H. Meadows, *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green Publishing, 2008.
- [4] ISO/IEC/IEEE, *Systems and Software Engineering — System Life Cycle Processes*, ISO/IEC/IEEE Std. 15288:2015, 2015.
- [5] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.
- [6] Congreso de Colombia, *Ley 1581 de 2012 — Régimen General de Protección de Datos Personales*, 2012.
- [7] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2013.
- [8] N. Dragoni *et al.*, “Microservices: Yesterday, Today, and Tomorrow,” *Present and Ulterior Software Engineering*, pp. 195–216, 2017.